

React Redux Toolkit Counter

This project demonstrates how to build a typed Redux store with Redux Toolkit and connect it to React using `react-redux`.

The application contains a counter that can be incremented synchronously, decremented, or incremented asynchronously after a simulated one-second delay.

What Redux Toolkit solves

Redux provides a predictable way to manage shared application state. Redux Toolkit (RTK) is the recommended Redux approach because it reduces boilerplate and provides sensible defaults for:

- creating the store
- defining reducers and actions
- handling immutable updates
- writing asynchronous logic
- configuring Redux middleware and developer tools

This project uses RTK instead of manually writing action type constants, action creators, and switch-based reducers.

Dependencies

```
npm install @reduxjs/toolkit react-redux
```

- `@reduxjs/toolkit` provides `configureStore`, `createSlice`, and `createAsyncThunk`.
- `react-redux` provides the React `<Provider>` and hooks such as `useSelector` and `useDispatch`.

Project data flow

```
User clicks a button
  |
  v
Component dispatches an action
  |
  v
Counter slice reducer updates the state
  |
  v
Store publishes the new state
  |
  v
useSelector re-renders Counter with the new value
```

1. Create the slice

The counter slice is located in `src/state/counter/CounterSlice.ts`. A slice combines a feature's initial state, reducers, and generated action creators.

```
const counterSlice = createSlice({
  name: "counter",
  initialState: { value: 0 },
  reducers: {
    increment: (state) => {
      state.value += 1;
    },
    decrement: (state) => {
      state.value -= 1;
    },
    incrementByAmount: (state, action: PayloadAction<number>) => {
      state.value += action.payload;
    },
  },
});
```

Why direct mutation is safe here

The reducers appear to mutate `state.value` directly. Redux Toolkit uses Immer internally, so these updates are converted into safe immutable state updates. This gives the code a simple mutable style while preserving Redux's immutability requirements.

`createSlice` also generates action creators automatically:

```
export const { increment, decrement, incrementByAmount } = counterSlice.actions;
```

The generated action types are based on the slice name, such as `counter/decrement`.

2. Configure the store

The store is created in `src/state/store.ts`:

```
export const store = configureStore({
  reducer: {
    counter: counterReducer,
  },
});
```

The `counter` key determines the shape of the global state:

```
{
  counter: {
    value: 0,
  },
}
```

Because the reducer is registered under `counter`, components read the value using `state.counter.value`.

3. Add TypeScript types

The store derives the application's state and dispatch types from the configured store:

```
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

- `RootState` describes the complete Redux state tree.
- `AppDispatch` describes the dispatch function, including thunk support.

These types prevent selectors and dispatched actions from drifting away from the actual store configuration.

4. Provide the store to React

In `src/main.tsx`, the Redux `<Provider>` makes the store available to every component below it:

```
createRoot(document.getElementById("root")!).render(
  <Provider store={store}>
    <App />
  </Provider>,
);
```

Without the provider, `useSelector` and `useDispatch` would not know which Redux store to use.

5. Read and update state in a component

`src/components/Counter.tsx` reads the current value with `useSelector`:

```
const count = useSelector((state: RootState) => state.counter.value);
```

The component dispatches actions with a typed dispatch function:

```
const dispatch = useDispatch<AppDispatch>();

dispatch(decrement());
dispatch(incrementAsync(10));
```

When the reducer changes the state, the selector detects the update and React re-renders the counter display.

Synchronous actions

The slice contains three synchronous reducers:

- `increment` increases the value by one.
- `decrement` decreases the value by one.
- `incrementByAmount` increases the value by the numeric payload.

For example, `incrementByAmount(10)` creates an action whose payload is `10`, and the reducer adds that payload to the current value.

Asynchronous actions with `createAsyncThunk`

The project also demonstrates asynchronous Redux logic:

```
export const incrementAsync = createAsyncThunk(
  "counter/incrementAsync",
  async (amount: number) => {
    await new Promise((resolve) => setTimeout(resolve, 1000));
    return amount;
  },
);
```

`createAsyncThunk` automatically creates lifecycle actions for the async operation:

- `pending` when the operation starts
- `fulfilled` when it resolves successfully
- `rejected` when it fails

The slice handles these lifecycle actions in `extraReducers` :

```
extraReducers: (builder) => {
  builder
    .addCase(incrementAsync.pending, () => {
      console.log("incrementAsync.pending");
    })
    .addCase(incrementAsync.fulfilled, (state, action) => {
      state.value += action.payload;
    });
},
```

The counter changes only when the async action is fulfilled. In a production application, the pending and rejected cases could also update loading and error fields in the slice state.

Why `extraReducers` is used

The reducers inside `reducers` define actions owned by the slice. `extraReducers` responds to actions created somewhere else, such as the lifecycle actions generated by `createAsyncThunk` .

This keeps async state transitions in the same feature slice without manually creating separate action types.

Redux Toolkit compared with local state

Local React state is usually enough for state that belongs to one component, such as whether a menu is open or the current value of an input.

Redux is useful when state:

- is shared by multiple distant components
- needs one predictable update path
- represents application-level data
- requires reusable synchronous or asynchronous actions

- should be inspected through Redux DevTools

The counter is intentionally small, but the same structure scales to features such as authentication, shopping carts, filters, and multi-step workflows.

Key files

File	Responsibility
src/state/store.ts	Creates the Redux store and exports application types
src/state/counter/CounterSlice.ts	Defines counter state, reducers, actions, and async behavior
src/main.tsx	Provides the store to the React tree
src/components/Counter.tsx	Selects state and dispatches actions
src/App.tsx	Renders the counter feature

Run the project

```
npm install
npm run dev
```

To validate the TypeScript project and create a production build:

```
npm run build
```

Summary

This project demonstrates the essential Redux Toolkit workflow:

1. Define feature state with `createSlice` .
2. Register the slice reducer with `configureStore` .
3. Wrap the React tree with `<Provider>` .
4. Read state using `useSelector` .
5. Dispatch generated actions using `useDispatch` .
6. Handle asynchronous work with `createAsyncThunk` and `extraReducers` .

Redux Toolkit keeps global state explicit and predictable while making the setup concise enough for everyday React development.